

Maximum Likelihood in Machine Learning: Losses and Priors

Contents

1. Overview	2
2. A model as a distribution of the target	3
3. Gaussian noise gives least squares	4
3.1 The model	4
3.2 From the NLL to squared errors	4
3.3 The same answer as least squares	5
3.4 Estimating the noise	5
4. A Bernoulli target gives the log loss	6
5. A categorical target gives the cross entropy	7
6. Maximum likelihood overfits	7
7. MAP estimation: maximum likelihood plus a prior	9
7.1 The posterior over the parameters	9
7.2 A Gaussian prior gives ridge regression	9
7.3 A Laplace prior gives the lasso	10
8. Summary	12
9. Sources	12
10. Key terms	12

Where this fits: Pipeline map steps 0 (Foundations), 8 (Model). See the [Course map](#).



- **Builds on:** One-hot encoding ([Note 27](#)); Softmax regression ([Note 79](#)); Overfitting ([Note 91](#)); Bayes' theorem ([Note 341](#)); Derivatives of one variable ([Note 600](#)); Partial derivatives and gradients ([Note 601](#)).
- **Leads to:** Gaussian mixture model (GMM) ([Note 640](#)); Expectation maximization (EM) ([Note 641](#)); ANN for classification ([Note 1011](#)).
- **Compare with:** Hinge loss and soft margin ([Note 94](#)); Entropy, information gain and Gini ([Note 97](#)).

1. Overview

Key point: The usual ML losses are negative log-likelihoods. Gaussian noise gives the mean squared error, a Bernoulli target gives the log loss, a categorical target gives the cross entropy. Adding a prior on the parameters (MAP) gives regularisation.

This Note follows *Mathematics for Machine Learning* (Deisenroth, Faisal and Ong, 2020; MML below), Section 8.3 (Parameter Estimation) and Section 9.2 (Parameter Estimation for linear regression).

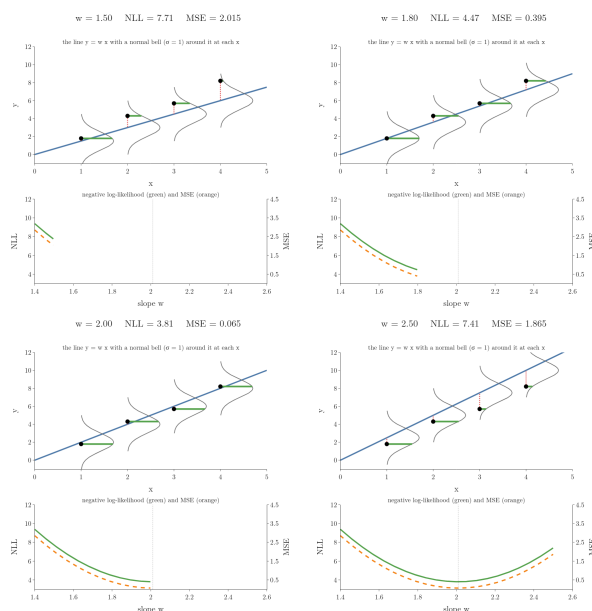


Figure 1: A line $y = wx$ through four points, with the slope sweeping from 1.4 to 2.6. Each grey bell is the normal distribution of y around the line; the green bar is its height at the observed y . Bottom: the negative log-likelihood and the mean squared error against w , lowest at the same slope

The [maximum likelihood estimation Note](#) fitted one distribution to a column of numbers. In ML we predict a **target** y (the output we want) from **features** x (the input variables, one column each of the data table). One record, the features of one case together with its target, is an **observation** (one row of the data table). The distribution of y changes with x . Figure 1 shows the idea for regression: around the line sits a bell, and a good line puts every point near the top of its bell.

Figure 2 is the map of this Note. Each model of the target turns, through maximum likelihood, into a loss we already know; each prior on the parameters turns, through MAP estimation, into a penalty we already know.

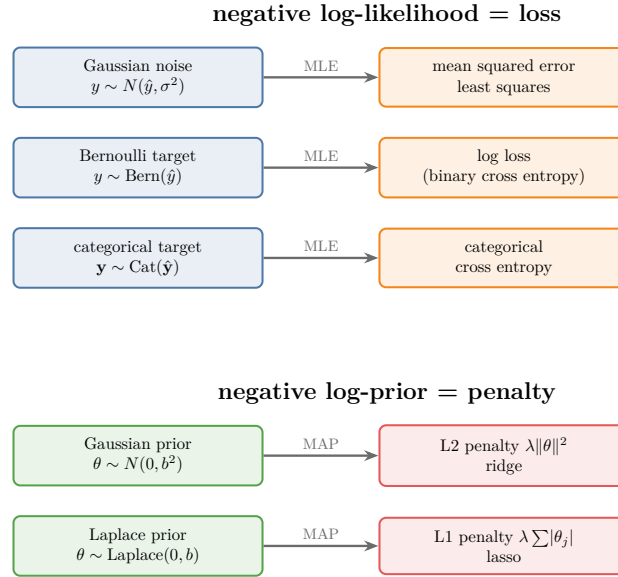


Figure 2: Models of the target become losses under maximum likelihood; priors on the parameters become penalties under MAP

2. A model as a distribution of the target

Key point: A probabilistic model does not predict one value for y ; it gives a whole distribution $p(y \mid x, \theta)$. The likelihood of the training set is the product of these over all observations.

A model with parameters θ (all the weights) takes the features x and returns a distribution over the possible targets. We write it as a conditional distribution (see the [joint, marginal and conditional probability Note](#)):

$$p(y \mid x, \theta)$$

Training observations $(x_1, y_1), \dots, (x_n, y_n)$ are assumed i.i.d., so the likelihood is a product over observations and the negative log-likelihood (NLL) is a sum:

1. **In words:** the NLL adds, over the training observations, minus the log of the probability (or density) the model gives to each observation's true target.
2. **Formula:**

$$\text{NLL}(\theta) = - \sum_{i=1}^n \log p(y_i | x_i, \theta), \quad \hat{\theta}_{\text{ML}} = \arg \min_{\theta} \text{NLL}(\theta)$$

3. **Example:** if a model gives the true targets of three observations probabilities 0.9, 0.5 and 0.8, then $\text{NLL} = -(\log 0.9 + \log 0.5 + \log 0.8) = 0.105 + 0.693 + 0.223 = 1.02$.

Two choices define a model: how the prediction depends on x (a line, a sigmoid, a neural network) and which distribution describes the target around that prediction. The second choice decides the loss.

Extra: MML (§8.3.1) warns that θ standing to the right of the bar in $p(y | x, \theta)$ does not make it fixed. In the NLL, the data is fixed and θ is the variable, exactly as in the [probability vs likelihood Note](#).

3. Gaussian noise gives least squares

Key point: If y is the prediction plus normal noise, the NLL is the sum of squared errors divided by $2\sigma^2$, plus a constant. Minimising it is least squares.

3.1 The model

Key point: $y = \hat{y} + \varepsilon$ with $\varepsilon \sim N(0, \sigma^2)$: the target is normal around the prediction.

Linear regression predicts $\hat{y} = wx + b$ (see the [simple linear regression Note](#)). The points never lie exactly on the line. We model the gap as **noise**: a random error ε drawn from a normal distribution with mean 0 and standard deviation σ (see the [normal distribution Note](#)). Then

$$p(y | x, \theta) = N(y | \hat{y}, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(y-\hat{y})^2}{2\sigma^2}}$$

In Figure 1 each grey bell is this distribution at one x . Its peak sits on the line, and its height at the observed y is that observation's likelihood (green bar). The bell model is the “normality of residuals” assumption of the [linear regression assumptions Note](#), now used to derive the loss.

3.2 From the NLL to squared errors

Key point: The log of a normal density is a constant minus $(y - \hat{y})^2 / (2\sigma^2)$. So the NLL is the squared error plus a constant.

1. **In words:** the log of the normal density splits into a part that does not depend on the weights and minus the squared error over $2\sigma^2$ (as in the [MLE for common distributions Note](#), Section 4.2). With σ fixed, only the squared errors change with the weights.
2. **Formula:**

$$\text{NLL}(\theta) = \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + n \log(\sigma\sqrt{2\pi})$$

The second term is a constant, and the factor $1/(2\sigma^2)$ only rescales the first. So the θ with the smallest NLL is the θ with the smallest sum of squared errors, and also the smallest mean squared error.

3. **Example:** four points $x = 1, 2, 3, 4$ with $y = 1.8, 4.3, 5.7, 8.2$, the line $\hat{y} = wx$ and $\sigma = 1$. At $w = 2$ the errors are $-0.2, 0.3, -0.3, 0.2$, with squared sum 0.26:

$$\text{NLL}(2) = \frac{0.26}{2} + 4 \times 0.919 = 0.13 + 3.676 = 3.81, \quad \text{MSE}(2) = \frac{0.26}{4} = 0.065$$

At $w = 2.5$: NLL 7.41 and MSE 1.865. Both are worse.

The bottom panel of Figure 1 draws both curves. They have different heights, but their lowest points are at the same slope.

3.3 The same answer as least squares

Key point: Setting the derivative to 0 gives the ordinary least squares formula; for many features, the normal equation.

For the line through the origin, the derivative of $\sum (y_i - wx_i)^2$ with respect to w is $-2 \sum x_i(y_i - wx_i)$. Setting it to 0:

$$\hat{w} = \frac{\sum_i x_i y_i}{\sum_i x_i^2} = \frac{1.8 + 8.6 + 17.1 + 32.8}{1 + 4 + 9 + 16} = \frac{60.3}{30} = 2.01$$

With an intercept and many features, the same derivation gives the normal equation $\hat{\beta} = (X^T X)^{-1} X^T y$ of the [multiple linear regression maths Note](#). So ordinary least squares, the mean squared error of the [regression metrics Note](#) and maximum likelihood with Gaussian noise all give the same line.

The NLL here is a quadratic bowl in the weights. MML (§9.2.1, remark after equation 9.12) shows that its Hessian, $X^T X$, is positive definite, so the single flat point is the global minimum (convexity: the [convex and non-convex cost functions Note](#)).

3.4 Estimating the noise

Key point: Maximum likelihood also estimates σ^2 : it is the mean squared residual of the fitted line.

Treating σ as a parameter too and setting its derivative to 0 (MML §9.2.1, equation 9.22) gives the same result as the normal MLE variance of the [MLE for common distributions Note](#), with the residuals in place of the distances from the mean:

1. **In words:** the MLE of the noise variance is the average squared distance between the targets and the fitted line.
2. **Formula:**

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

3. **Example:** at $\hat{w} = 2.01$ the squared residuals add up to 0.257, so $\hat{\sigma}^2 = 0.257/4 = 0.064$ and $\hat{\sigma} = 0.25$. The points scatter about 0.25 above and below the line.

So the training MSE of a least squares fit is the maximum likelihood estimate of the noise variance.

Extra: Choosing a different noise distribution gives a different loss. With **Laplace noise**, density $e^{-|y-\hat{y}|/b}/(2b)$, the log of one density is $-\log(2b) - |y - \hat{y}|/b$. Summing and changing the sign, the NLL is $\sum_i |y_i - \hat{y}_i|/b + n \log(2b)$: with b fixed, minimising it minimises the absolute errors, the mean absolute error of the [regression metrics Note](#), called the L1 loss in the [loss functions Note](#). The Notebook checks this by minimising the Laplace NLL numerically. The Laplace curve has heavier tails than the normal curve, so a far-away point is less surprising under it; in loss terms, an outlier adds its distance, not its squared distance. The heavier tails are why the absolute-error loss is more robust to outliers (Murphy §7.4).

4. A Bernoulli target gives the log loss

Key point: With a 0/1 target and predicted probability $\hat{y}$, the Bernoulli PMF $\hat{y}^y(1 - \hat{y})^{1-y}$ has minus log $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$: the log loss term for one observation.

The [log loss Note](#) built the binary cross entropy by multiplying the probabilities of the true classes and taking minus the log. That Note found the formula $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$ by noticing that the y factors switch the right term on. The NLL view shows where that formula comes from.

1. **In words:** model the target as a Bernoulli trial (see the [Bernoulli and binomial Note](#), Section 2.1) whose success probability is the sigmoid output $\hat{y} = \sigma(w \cdot x)$. Minus the log of its PMF is the log loss term.
2. **Formula:**

$$p(y | x, \theta) = \hat{y}^y(1 - \hat{y})^{1-y} \implies -\log p(y | x, \theta) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

The log brings the exponents y and $1 - y$ down as multipliers. Summed over the observations, this is the binary cross entropy; divided by n , the log loss.

3. **Example:** the four observations of the log loss Note, model 1: targets 1, 0, 1, 0 with $\hat{y} = 0.7, 0.6, 0.4, 0.2$. The Bernoulli PMF gives $0.7^1 0.3^0 = 0.7$, then $0.6^0 0.4^1 = 0.4$, then 0.4 and 0.8. The NLL is $-\log(0.7 \times 0.4 \times 0.4 \times 0.8) = 2.41$, the cross entropy found there.

So logistic regression is maximum likelihood estimation for a Bernoulli model whose probability comes from a sigmoid. Unlike Gaussian noise, the weights sit inside the sigmoid and the logs, so there is no closed form and we use gradient descent.

5. A categorical target gives the cross entropy

Key point: With K classes and one-hot target $\mathbf{y}$, the categorical PMF $\prod_k \hat{y}_k^{y_k}$ has minus $\log - \sum_k y_k \log \hat{y}_k$: the categorical cross entropy.

The **categorical distribution** is the Bernoulli distribution extended to K outcomes: outcome k has probability $\hat{y}_k$, and the $\hat{y}_k$ add up to 1. A softmax output provides exactly such probabilities (see the [softmax regression Note](#)).

1. **In words:** write the target one-hot (1 for the true class, 0 elsewhere). The probability of the true class is the product of $\hat{y}_k^{y_k}$, because every factor with $y_k = 0$ equals 1. Minus its log is the categorical cross entropy.
2. **Formula:**

$$p(\mathbf{y} \mid x, \theta) = \prod_{k=1}^K \hat{y}_k^{y_k} \implies -\log p(\mathbf{y} \mid x, \theta) = -\sum_{k=1}^K y_k \log \hat{y}_k$$

3. **Example:** three classes, softmax output (0.7, 0.2, 0.1), true class the first, $\mathbf{y} = (1, 0, 0)$. The product is $0.7^1 \times 0.2^0 \times 0.1^0 = 0.7$, and the loss is $-\log 0.7 = 0.357$.

The categorical cross entropy is the loss of softmax regression and of every classification network in the [loss functions Note](#). Training a classifier by minimising cross entropy is maximum likelihood estimation.

Model of the target	NLL per observation	Name of the loss
$N(\hat{y}, \sigma^2)$, any real y	$(y - \hat{y})^2 / (2\sigma^2) + \text{const}$	squared error (MSE)
Laplace($\hat{y}, b$)	$ y - \hat{y} /b + \text{const}$	absolute error (MAE)
Bern($\hat{y}$), $y \in \{0, 1\}$	$-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$	log loss
Cat($\hat{y}_1, \dots, \hat{y}_K$), one-hot $\mathbf{y}$	$-\sum_k y_k \log \hat{y}_k$	categorical cross entropy

6. Maximum likelihood overfits

Key point: Maximum likelihood uses all the freedom a model has to match the training data, noise included. With many parameters and few observations, the training NLL keeps falling while the error on new data explodes.

A model that is too flexible learns the noise of its training data and fails on new data: this is **overfitting** (see the [challenges in ML Note](#) and the [bias-variance Note](#)). Maximum likelihood has no brake against overfitting. The method is told to make the observed data as likely as possible, and a model that passes through every point does that best.

Figure 3 repeats the experiment of MML §9.2.2, with our own random draw of the data (Notebook, Section 4). Ten points come from $y = -\sin(x/5) + \cos(x)$ plus noise with $\sigma = 0.2$. We fit polynomials of degree 0 to 9 by maximum likelihood with Gaussian noise, which is least squares on polynomial features (see the [polynomial regression Note](#)).

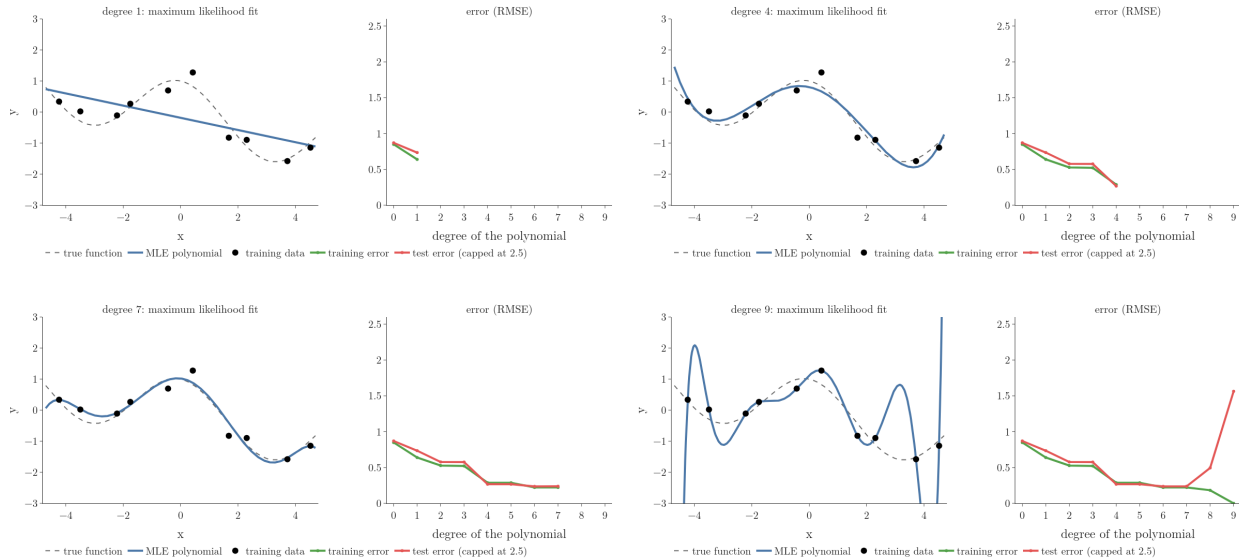


Figure 3: Maximum likelihood polynomials of degree 0 to 9 on ten points. Right: the training error falls to 0 while the test error, after a minimum, shoots up

Degree	0	1	2	4	6	8	9
Training RMSE	0.85	0.64	0.53	0.29	0.22	0.18	0.00
Test RMSE (200 new points)	0.87	0.74	0.58	0.27	0.24	0.49	1.56

- The **training error never rises** as the degree grows (the book observes the same). The reason is a one-line argument: every polynomial of degree $M - 1$ is also a polynomial of degree M with last coefficient 0, so the best degree M fit can do no worse on the training data.
- At degree 9 there are 10 coefficients for 10 points. The polynomial passes through every point, the training error is 0, and the curve swings wildly between the points.
- The **test error** is lowest around degree 6 and then rises fast.

With more parameters than observations the MLE is not even unique (MML §9.2.2): the matrix $X^T X$ in the normal equation cannot be inverted, and infinitely many coefficient vectors fit the data equally well. The next section adds the missing brake.

7. MAP estimation: maximum likelihood plus a prior

Key point: MAP estimation maximises likelihood $\times$ prior. Its negative log is the NLL plus a penalty on the parameters: a Gaussian prior gives the ridge penalty, a Laplace prior the lasso penalty.

7.1 The posterior over the parameters

Key point: Bayes' theorem with the parameters as the unknown: posterior $\propto$ likelihood $\times$ prior.

MML (§9.2.3) observes that parameter values often become large when a model overfits, and proposes to state beforehand which values are plausible. A **prior** $p(\theta)$ encodes that belief as a distribution over the parameters. Bayes' theorem (see the [Bayes' theorem Note](#)) combines it with the likelihood into a **posterior**:

$$p(\theta \mid \text{data}) = \frac{p(\text{data} \mid \theta) p(\theta)}{p(\text{data})}$$

The evidence $p(\text{data})$ does not depend on θ , so for finding the best θ we can drop it, as Naive Bayes dropped it when comparing classes (the [Naive Bayes intuition Note](#)).

1. **In words:** the **maximum a posteriori (MAP)** estimate is the θ with the largest posterior. In logs, it minimises the NLL plus minus the log of the prior.
2. **Formula:**

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\text{data} \mid \theta) p(\theta) = \arg \min_{\theta} [\text{NLL}(\theta) - \log p(\theta)]$$

3. **Example:** with a flat prior, the same for every θ , $-\log p(\theta)$ is a constant and MAP gives the MLE. The prior only matters when it prefers some values over others.

The [Naive Bayes maths Note](#) used the MAP rule to pick a class. Here the same principle picks parameter values.

7.2 A Gaussian prior gives ridge regression

Key point: With prior $\theta_j \sim N(0, b^2)$ and Gaussian noise σ , MAP minimises $\sum (y_i - \hat{y}_i)^2 + \lambda \sum_j \theta_j^2$ with $\lambda = \sigma^2/b^2$.

1. **In words:** a normal prior centred on 0 says each weight is probably small. Minus its log is the squared weight over $2b^2$, plus a constant. Added to the Gaussian NLL and multiplied by $2\sigma^2$, this is the ridge loss of the [ridge regression maths Note](#).
2. **Formula** (MML equations 9.28 and 9.33):

$$\text{NLL}(\theta) - \log p(\theta) = \frac{1}{2\sigma^2} \sum_i (y_i - \hat{y}_i)^2 + \frac{1}{2b^2} \sum_j \theta_j^2 + \text{const} \propto \sum_i (y_i - \hat{y}_i)^2 + \frac{\sigma^2}{b^2} \sum_j \theta_j^2$$

So $\lambda = \sigma^2/b^2$: a narrow prior (small b) or noisy data (large σ) means strong regularisation.

3. **Example:** the four points of Section 3 with $\sigma = 1$ and prior $w \sim N(0, 0.5^2)$, so $\lambda = 1/0.25 = 4$. As in the ridge maths Note, λ is added to the bottom of the fraction:

$$\hat{w}_{\text{MAP}} = \frac{\sum_i x_i y_i}{\sum_i x_i^2 + \lambda} = \frac{60.3}{30 + 4} = 1.77$$

The prior pulls the slope from the MLE 2.01 towards 0.

Figure 4 applies this to the degree-9 polynomial of Figure 3, with prior $N(0, 0.1^2)$ on every coefficient and $\sigma = 0.2$, so $\lambda = 0.04/0.01 = 4$. The MLE swings through every point; the MAP curve stays close to the true function, and its test RMSE drops from 1.56 to 0.54.

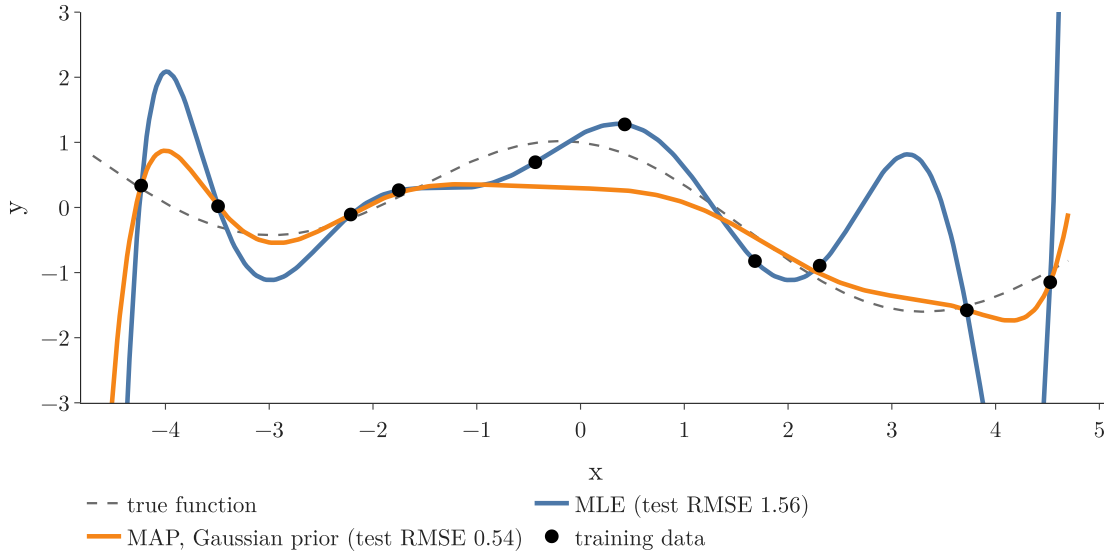


Figure 4: Degree-9 polynomial on ten points: maximum likelihood versus MAP with a Gaussian prior

7.3 A Laplace prior gives the lasso

Key point: With a Laplace prior, minus the log prior is $\sum_j |\theta_j|/b$: the L1 penalty of the lasso.

1. **In words:** the Laplace prior has a sharp peak at 0, so it believes many weights are exactly 0. Minus its log is the absolute value of each weight divided by b .
2. **Formula:** with $p(\theta_j) = e^{-|\theta_j|/b}/(2b)$,

$$\text{NLL}(\theta) - \log p(\theta) \propto \sum_i (y_i - \hat{y}_i)^2 + \frac{2\sigma^2}{b} \sum_j |\theta_j|$$

The result is the loss of the [lasso regression Note](#), with $\lambda = 2\sigma^2/b$.

3. **Example:** with $\sigma = 1$ and $b = 0.5$, $\lambda = 2/0.5 = 4$; a weight of 0.3 costs a penalty of $4 \times 0.3 = 1.2$.

MML (§9.5) states this equivalence of the Laplace prior and the lasso. The term $|\theta_j|/b$ has a corner at 0, the same corner of $|m|$ that lets lasso coefficients reach exactly 0 in the [lasso sparsity Note](#).

Extra: MAP still returns a single **point estimate**, one set of parameter values (MML §9.2.4). The book notes (Section 9.2.3, Example 9.6) that a prior can push back overfitting but is not a general cure, and continues in Section 9.3 with Bayesian linear regression, which keeps the whole posterior distribution and averages the predictions of all plausible parameter values.

8. Summary

Ingredient	Becomes	Example in this Note
Gaussian noise + MLE	least squares, MSE	slope $\hat{w} = 60.3/30 = 2.01$
MLE of the noise variance	mean squared residual	$\hat{\sigma}^2 = 0.064$
Laplace noise + MLE	MAE (L1 loss)	
Bernoulli target + MLE	log loss	NLL 2.41 for model 1
Categorical target + MLE	categorical cross entropy	$-\log 0.7 = 0.357$
Gaussian prior + MAP	ridge, $\lambda = \sigma^2/b^2$	$\hat{w} = 60.3/34 = 1.77$
Laplace prior + MAP	lasso, $\lambda = 2\sigma^2/b$	

- A probabilistic model gives a distribution $p(y \mid x, \theta)$ over the target; training minimises its NLL.
- The choice of distribution for the target decides the loss: normal gives squared error, Bernoulli gives log loss, categorical gives cross entropy.
- Maximum likelihood overfits flexible models: the training error never rises with more parameters.
- MAP adds $-\log p(\theta)$ to the NLL; a Gaussian prior is ridge, a Laplace prior is lasso.

9. Sources

Built from

- Deisenroth, M. P., Faisal, A. A. and Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. Free PDF at mml-book.github.io. §8.3.1 (Gaussian likelihood gives least squares, eq. 8.18), §8.3.2 (MAP), §9.2.1 (MLE for linear regression, Hessian remark, noise variance eq. 9.22), §9.2.2 (overfitting experiment), §9.2.3–9.2.4 (MAP as regularisation, eqs. 9.28 and 9.33), §9.5 (Laplace prior and lasso).

Other references

- Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press. §7.4 (robust linear regression with the Laplace likelihood).
- scikit-learn documentation: **LogisticRegression** ($C = \text{inf}$ for no penalty) and **Ridge**, used in the Notebook checks.

10. Key terms

Term	Meaning
Probabilistic model	A model that outputs a distribution $p(y \mid x, \theta)$ over the target rather than a single value
Noise (ε)	The random part of a target that the model's prediction does not explain

Term	Meaning
Gaussian noise	Noise drawn from $N(0, \sigma^2)$; under MLE it gives the squared-error loss
Laplace distribution	A peaked, heavy-tailed distribution with density $e^{- x-\mu /b}/(2b)$
Categorical distribution	The distribution of one draw among K classes with probabilities adding up to 1
Prior $p(\theta)$	A distribution over the parameters expressing what we believe before seeing data
Posterior $p(\theta \mid \text{data})$	The distribution over the parameters after seeing the data; proportional to likelihood $\times$ prior
Maximum a posteriori (MAP) estimation	Choosing the parameters with the largest posterior: minimise NLL minus the log prior
Point estimate	A single set of parameter values, as returned by MLE and MAP